8.1.1. Graph implementation - Adjacency list Directed graph

Code :

```
#include <stdio.h>

#include <stdlib.h>


struct node {

   int vertex;

   struct node *next;

};


typedef struct node* GNODE;

GNODE graph[20]; // Adjust size as per your system limits


void print(int *N) {

   for (int i = 1; i <= *N; i++) {

      if (graph[i] != NULL) {

         printf("%d=>", i);

         GNODE temp = graph[i];

         while (temp != NULL) {

            printf("%d\t", temp->vertex);

            temp = temp->next;

         }

         printf("\n");

      }

   }

}


void insertVertex(int *N) {
```

```
int numIn, numOut, s, d;
(*N)++;
int newV = *N;
graph[newV] = NULL;

printf("Enter the number of edges from existing vertices to the new vertex: ");
scanf("%d", &numIn);
for (int i = 0; i < numIn; i++) {
   printf("Enter source : ");
   scanf("%d", &s);
   if (s < 1 || s >= newV) {
      printf("Invalid vertex.\n");
   } else {
      GNODE q = (GNODE)malloc(sizeof(struct node));
      q->vertex = newV;
      q->next = NULL;
      if (graph[s] == NULL) graph[s] = q;
      else {
         GNODE p = graph[s];
         while (p->next != NULL) p = p->next;
         p->next = q;
      }
   }
}

printf("Enter the number of edges from the new vertex to existing vertices: ");
scanf("%d", &numOut);
for (int i = 0; i < numOut; i++) {
```

```
        printf("Enter destination : ");
        scanf("%d", &d);
        if (d < 1 || d >= newV) {
            printf("Invalid vertex.\n");
        } else {
            GNODE q = (GNODE)malloc(sizeof(struct node));
            q->vertex = d;
            q->next = NULL;
            if (graph[newV] == NULL) graph[newV] = q;
            else {
                GNODE p = graph[newV];
                while (p->next != NULL) p = p->next;
                p->next = q;
            }
        }
    }
    printf("After inserting vertex the adjacency list is : \n");
    print(N);
}

void insertEdge(int *N) {
    int s, d;
    printf("Enter the source vertex of the edge : ");
    scanf("%d", &s);
    printf("Enter the destination vertex of the edge : ");
    scanf("%d", &d);

    if (s < 1 || s > *N || d < 1 || d > *N) {
```

```
        printf("Invalid vertex.\n");
    } else {
        GNODE q = (GNODE)malloc(sizeof(struct node));
        q->vertex = d;
        q->next = NULL;
        if (graph[s] == NULL) graph[s] = q;
        else {
            GNODE p = graph[s];
            while (p->next != NULL) p = p->next;
            p->next = q;
        }
        printf("After inserting edge the adjacency list is : \n");
        print(N);
    }
}

void deleteVertex(int *N) {
    if (*N == 0) {
        printf("Graph is empty.\n");
        return;
    }
    int v;
    printf("Enter the vertex to be deleted : ");
    scanf("%d", &v);
    if (v < 1 || v > *N) {
        printf("Invalid vertex.\n");
        return;
    }
```

```
// Free the list for vertex v
GNODE temp = graph[v];
while (temp != NULL) {
   GNODE toFree = temp;
   temp = temp->next;
   free(toFree);
}
graph[v] = NULL;

// Remove incoming edges from other vertices
for (int i = 1; i <= *N; i++) {
   GNODE curr = graph[i], prev = NULL;
   while (curr != NULL) {
      if (curr->vertex == v) {
         if (prev == NULL) graph[i] = curr->next;
         else prev->next = curr->next;
         GNODE toFree = curr;
         curr = curr->next;
         free(toFree);
      } else {
         prev = curr;
         curr = curr->next;
      }
   }
}
printf("After deleting vertex the adjacency list is : \n");
print(N);
```

```
}

void deleteEdge(int *N) {
  int s, d;
  printf("Enter the source vertex of the edge : ");
  scanf("%d", &s);
  printf("Enter the destination vertex of the edge : ");
  scanf("%d", &d);

  if (s < 1 || s > *N || d < 1 || d > *N) {
    printf("Invalid vertex.\n");
    return;
  }

  GNODE curr = graph[s], prev = NULL;
  while (curr != NULL) {
    if (curr->vertex == d) {
      if (prev == NULL) graph[s] = curr->next;
      else prev->next = curr->next;
      free(curr);
      break;
    }
    prev = curr;
    curr = curr->next;
  }
  printf("After deleting edge the adjacency list is : \n");
  print(N);
}
```

Programme :

CODETANTRA Home

yash.chandak.batch2025@sitnagpur.siu.edu.in Support Logout

8.1.1. Graph implementation - Adjacency list Directed graph 00:13

Write a C program to represent a Directed Graph using an adjacency list.

Functions to be implemented in **GraphsAdjacencyListDirectedGraph.c** file:

1. void print(int *N) - This function should display the adjacency list of the directed graph.
- For each vertex that has outgoing edges, print the vertex followed by =>.
- Print all adjacent vertices separated by a tab character.
- Print each vertex's adjacency list on a new line.

2. void insertVertex(int *N) - This function should insert a new vertex into the graph.
- Read the number of edges from existing vertices to the new vertex
- For each such edge, read the source vertex number
  - An edge is added from the source vertex to the new vertex
- Read the number of edges from the new vertex to existing vertices
- For each such edge, read the destination vertex number
  - An edge is added from the new vertex to the destination vertex
- If an invalid vertex is entered, display:

```
Invalid vertex.
```

- After successful insertion, display:

```
After inserting vertex the adjacency list is :
```

and print the updated adjacency list.

3. void insertEdge(int *N) - This function should insert a directed edge from a source vertex to a

Explorer

GraphsA... Submit

```
142    int s, d;
143    printf("Enter the source vertex of the edge : ");
144    scanf("%d", &s);
145    printf("Enter the destination vertex of the edge : ");
146    scanf("%d", &d);
147
148    if (s < 1 || s > *N || d < 1 || d > *N) {
149        printf("Invalid vertex.\n");
150        return;
151    }
152
```

Average time 0.034 s 34.00 ms

Maximum time 0.045 s 45.00 ms

2 out of 2 shown test case(s) passed

1 out of 1 hidden test case(s) passed

Test case 1 33 ms Debug

Expected output	Actual output
Enter the number of vertices : 3	Enter the number of vertices : 3
Enter the number of edges : 3	Enter the number of edges : 3
Enter source : 1	Enter source : 1
Enter destination : 2	Enter destination : 2
Enter source : 1	Enter source : 1
Enter destination : 3	Enter destination : 3

Terminal Test cases

Debugger

< Prev Reset Submit Next >